

เบื้องหลังเว็บไซต์ PORTFOLIO

คู่มือวิธีสร้าง เทคนิคที่ใช้ และแนวทางอธิบายผลงานจากโค้ดจริง

โซติกา การเกิด (หน้าฝน)

ม.6 | แผนการเรียน CSE | โรงเรียนบ้านบึงอุตสาหกรรมนุเคราะห์
เป้าหมายการศึกษาต่อ: คณะสัตวแพทยศาสตร์ มหาวิทยาลัยมหิดล

แนวคิดของงาน

เว็บไซต์แฟ้มสะสมผลงาน 4 หน้า
เน้นอ่านง่าย ใช้โทนสีเงินกับฟ้าอ่อน
และปรับการจัดวางตามขนาดหน้าจอ

เทคโนโลยีหลัก

HTML + CSS + JavaScript
Node.js สำหรับเปิดเว็บในเครื่อง
Git และ GitHub Pages สำหรับเผยแพร่



ขอบเขตและที่มาของเอกสาร

สรุปจากไฟล์โครงการ rxin และประวัติการพัฒนาที่ตรวจสอบได้
ผู้ใช้กำหนดเนื้อหา รูปภาพ และทิศทางการปรับแก้ โดยมี AI ช่วยเขียนโค้ด
แก้ภาพพื้นหลัง และจัดทำเอกสารฉบับนี้ จึงควรนำเสนอว่าเป็นงานที่พัฒนา
โดยใช้ AI ช่วย และอธิบายเฉพาะส่วนที่ตนเข้าใจและได้ลงมือทดสอบจริง
อ้างอิงเวอร์ชันโค้ด: b2c4fd6 | เรียบเรียง 16 กันยายน 2569

01 วางแผนและแบ่งหน้าที่

เริ่มจากสิ่งที่ผู้อ่านต้องการเห็น ก่อนเลือกเทคนิคการเขียนโค้ด

โครงสร้าง 4 หน้า

1. index.html: หน้าปก PORTFOLIO ภาพบุคคล เป้าหมาย และคำนำ
2. profile.html: ชื่อ ชื่อเล่น ระดับชั้น แผนการเรียน เกรด และอีเมล
3. certificates.html: รายการเกียรติบัตร พร้อมกดเปิดภาพขนาดใหญ่
4. thanks.html: ข้อความขอบคุณและลิงก์กลับสู่หน้าแรก

เหตุผลที่ใช้เว็บแบบ Static

เนื้อหาชุดนี้เป็นข้อมูลพอร์ตที่แก้เป็นครั้งคราว จึงใช้ไฟล์ HTML, CSS และ JavaScript ได้โดยไม่ต้องมีฐานข้อมูลหรือระบบเข้าสู่ระบบ
GitHub Pages ส่งไฟล์เหล่านี้ให้เบราว์เซอร์ จากนั้น JavaScript จึงสร้างเนื้อหาของแต่ละหน้า เหมาะกับเว็บขนาดเล็กและดูแลต่อได้ง่าย

หน้าที่ของแต่ละภาษา

HTML: กำหนดโครงเอกสาร ภาษาไทย ชื่อหน้า และจัดวางเนื้อหา
CSS: กำหนดสี ตัวอักษร ระยะห่าง รูปภาพ และการแสดงผลบนมือถือ
JavaScript: อ่านข้อมูล เลือกหน้า สร้างรายการ และจัดการการกดภาพ
Node.js: ใช้เปิดเซิร์ฟเวอร์ในเครื่อง ไม่ได้ทำงานบน GitHub Pages

ลำดับทำงานที่นำไปทำตามได้

กำหนดหน้าที่ต้องมี > เตรียมไฟล์ > ใส่ข้อมูล > จัดหน้า > เพิ่มปฏิสัมพันธ์
จากนั้นตรวจในเครื่อง > commit > push > เผยแพร่ > ตรวจเว็บออนไลน์
ไม่จำเป็นต้องเริ่มจากระบบซับซ้อน ควรให้แต่ละส่วนทำงานได้ก่อน

02 อ่านโครงสร้างโครงการ

รู้ว่าแก้ไฟล์ใด และการเปลี่ยนแปลงส่งผลต่อหน้าไหน

```
rxin/  
dist/  
  index.html, profile.html  
  certificates.html, thanks.html  
  content.js  # ข้อมูลส่วนตัวและเกียรติบัตร  
  app.js      # สร้างหน้าและควบคุมปฏิสัมพันธ์  
  styles.css  # รูปแบบและ responsive  
  assets/     # รูปบุคคล โลโก้ และเกียรติบัตร  
  server.cjs  # เซิร์ฟเวอร์สำหรับดูในเครื่อง  
  validate.cjs # ตรวจสอบโครงสร้างและเส้นทางไฟล์
```

ข้อมูลถูกแยกออกจากส่วนแสดงผล

content.js เก็บข้อมูลใน window.PORTFOLIO เพียงจุดเดียว

app.js อ่านค่ามาใช้ทุกหน้า เช่น ชื่อที่ปรากฏทั้งหน้าปกและหน้าขอบคุณ

วิธีนี้ลดการแก้ข้อความซ้ำ และช่วยให้ข้อมูลแต่ละหน้าตรงกัน

```
nickname: 'น้ำฝน',  
grade: 'ม.6',  
program: 'CSE',  
gpa: '4.00',  
portrait: 'assets/CTK-blue.png',
```

ลำดับโหลดไฟล์สำคัญอย่างไร

HTML เรียก content.js ก่อน app.js และใส่ defer ทั้งสองไฟล์

เบราว์เซอร์จึงรออ่าน HTML เสร็จ แล้วทำงานตามลำดับที่ประกาศ

app.js จึงเข้าถึงทั้งข้อมูลและ <div id="app"> ได้เมื่อเริ่มทำงาน

03 เทคนิคออกแบบและมือถือ

ใช้สี ตัวอักษร และ CSS Grid ให้เนื้อหา มีลำดับชัดเจน

สีและลำดับตัวอักษร

สีหลัก #173d80 เป็นสีเงินที่เลือกใช้ในงานออกแบบนี้

ใช้พื้น #f8fafc และฟ้าอ่อน #e7eef7 ช่วยให้หน้าดูสบายตา

หัวข้อ PORTFOLIO ใหญ่กว่าชื่อบท ส่วนรายละเอียดใช้ขนาดเล็กลง

เว็บไซต์ใช้ DM Sans และ Noto Sans Thai ผ่าน Google Fonts

จากสองคอลัมน์เป็นคอลัมน์เดียว

```
.hero {  
  display: grid;  
  grid-template-columns: 1.16fr 1fr;  
}  
@media (max-width: 700px) {  
  .hero { grid-template-columns: 1fr; }  
}
```

บนจอกว้าง ข้อความกับรูปภาพข้างกัน เมื่อหน้าจอแคบลงจะเรียงบนล่าง
ในโค้ดยังมีจุดปรับที่ 1000px และใช้ clamp() ให้ขนาดหัวข้อยืดหยุ่น
ปุ่มหน้าถัดไปมีความสูงอย่างน้อย 56px และขยายเต็มแถวบนมือถือ

รูปภาพต้องไม่ยืดหรือบีบ

ภาพบุคคลใช้ object-fit: cover เพื่อเต็มกรอบ โดยอาจมีส่วนถูกครอบ

โลโก้ใช้ object-fit: contain เพื่อแสดงตราครบและรักษาสัดส่วน

CTK-blue.png เป็นภาพที่ใช้ AI ช่วยเปลี่ยนพื้นหลังจากรูปต้นฉบับ

ควรตรวจใบหน้าและรายละเอียดกับต้นฉบับก่อนใช้ในงานจริง

รายละเอียดเพื่อการใช้งาน

มีข้อความ alt สำหรับรูป เมนูระบุ aria-current และลิงก์ข้ามไปเนื้อหา

มีกรอบ focus-visible สำหรับคีย์บอร์ด และลด motion ตามการตั้งค่า

สิ่งเหล่านี้ช่วยการเข้าถึง แต่ยังไม่ใช่วิธีรับรองการผ่านมาตรฐานทั้งหมด

04 JavaScript และเกียรติบัตร

จากข้อมูลหนึ่งชุด ไปสู่หน้าที่กดใช้งานได้

เลือกเนื้อหาตาม URL

```
const current = location.pathname.split('/').pop()
|| 'index.html';
const page = Math.max(0,
  routes.findIndex(r => r[0] === current));
```

อ่านชื่อไฟล์ท้าย URL แล้วหา index ใน routes เพื่อเลือกเนื้อหาแต่ละ HTML ใช้ app.js ร่วมกัน จึงมีเมนูและท้ายหน้าที่สอดคล้องกัน
นี่คือไฟล์หลายหน้าที่สร้างเนื้อหาด้วย JavaScript ไม่ใช่ React

เพิ่มรายการโดยแก้ไขข้อมูล

```
certificates: [
  { title: 'ชื่อเกียรติบัตร',
    issuer: 'หน่วยงานที่ออก',
    year: '2569',
    image: 'assets/certificate-2.jpg' }
]
```

วางไฟล์ภาพให้ตรงชื่อ แล้วเพิ่ม object ใน certificates
app.js ใช้ map() แปลงแต่ละรายการเป็นปุ่มการ์ดเกียรติบัตร
เมื่อกด ระบบอ่าน data-index เพื่อเลือกรายการ ตั้ง src และ alt
จากนั้นเรียก showModal() ของ <dialog> เพื่อแสดงภาพใหญ่
ปิดได้ด้วยปุ่มกากบาทหรือ Escape; CER1 ปัจจุบันเป็นไฟล์ทดสอบ

ป้องกันข้อความถูกอ่านเป็น HTML

ฟังก์ชัน esc() แปลงอักขระพิเศษ เช่น < > & และเครื่องหมายคำพูด
ก่อนนำข้อความไปประกอบ HTML ส่วนชื่อใน dialog ใช้ textContent
อย่างไรก็ดี ระบบนี้ยังไม่ใช้ระบบรับข้อมูลสาธารณะ หากเพิ่มการอัปโหลด
หรือรับ URL จากภายนอก ต้องเพิ่มการตรวจชนิดไฟล์และ URL ด้วย

05 ทดสอบและแก้ปัญหาแคช

แยกให้ชัดเจนระหว่างตรวจโค้ด ตรวจไฟล์ และทดสอบด้วยผู้ใช้

เปิดและตรวจในเครื่อง

```
node server.cjs
# เปิด http://127.0.0.1:4173
node --check dist/app.js
node --check dist/content.js
node validate.cjs
```

server.cjs ส่งไฟล์ใน dist และกำหนดชนิดข้อมูลให้เบราว์เซอร์

เซิร์ฟเวอร์ผูกกับ 127.0.0.1 จึงเปิดได้เฉพาะเครื่องนั้น

validate.cjs จำลองการสร้างหน้า ตรวจสอบว่ามี main และไฟล์ที่อ้างอิงอยู่จริง

การตรวจนี้ไม่ครอบคลุมหน้าต่าง การกด dialog หรือการใช้งานบนมือถือจริง

ทำไม push แล้วบางครั้งยังเห็นหน้าเดิม

การ push กับการเผยแพร่เป็นคนละขั้น: ต้องรอ Pages สร้างเว็บเสร็จ

นอกจากนี้เบราว์เซอร์หรือ CDN อาจเก็บไฟล์เก่าไว้ในแคช

จึงเพิ่ม query version ที่ลิงก์ไฟล์ CSS และ JavaScript ตาม hash

เมื่อไฟล์เปลี่ยน ให้คำนวณ hash ใหม่และอัปเดตใน HTML ทั้ง 4 หน้า

```
<script src="content.js?v=NEW_HASH" defer> </script>
<link rel="stylesheet" href="styles.css?v=NEW_HASH">
```

NEW_HASH เป็นตัวอย่าง ให้แทนด้วยค่าใหม่จริง ไม่ใช่ใช้ข้อความนี้คงที่

รูปพื้นหลังใหม่เปลี่ยนชื่อเป็น CTK-blue.png เพื่อแยกจากรูปเดิมด้วย

ผลที่ยืนยันได้ และสิ่งที่ยังต้องตรวจ

ยืนยันแล้ว: syntax, เส้นทางไฟล์, Pages สถานะ built และ HTTP 200

ตรวจพบข้อผิดพลาดและภาพใหม่ในไฟล์ออนไลน์หลังการเผยแพร่

ยังไม่ได้ยืนยันครบ: การแสดงผลในเบราว์เซอร์และมือถือจริง

ควรลองทุกเมนู เปิด-ปิดเกียร์ติบ์ตร ชุมข้อความ และเช็กการเลื่อนแนวนอน

06 Git และการเผยแพร่

เก็บโค้ดใน main และเผยแพร่เฉพาะ dist ผ่าน gh-pages

ทำความเข้าใจ branch ที่ใช้อยู่

main เก็บโครงการทั้งหมด เช่น server.cjs และไฟล์ dist

gh-pages เก็บเนื้อหาจาก dist โดยย้ายมาอยู่ราก branch

ตั้ง GitHub Pages ให้ใช้ branch gh-pages และโฟลเดอร์ /

จึงไม่ต้องรัน Node.js บนบริการ Pages

```
git status
git add dist
git commit -m "Update portfolio content"
git push origin main
git subtree split --prefix dist -b gh-pages
git push origin gh-pages
```

คำสั่งนี้เป็นลำดับที่ใช้กับโครงการนี้ และควรรันจากโฟลเดอร์ rxin

เครื่องต้องมี Git และยืนยันตัวตนกับ GitHub แล้ว

ก่อน commit ตรวจสอบ diff เพื่อไม่ส่งไฟล์ส่วนตัวหรือไฟล์ที่ไม่เกี่ยวข้อง

หากใช้ terminal ที่หา git ไม่เจอ ต้องใช้ตำแหน่ง git.exe ที่ติดตั้งไว้

ตรวจสอบหลังเผยแพร่ทุกครั้ง

1. รอ GitHub Pages แจ้งว่าเผยแพร่สำเร็จ
2. เปิดหน้าแรกและอีก 3 หน้า พร้อมตรวจสอบรูปและข้อมูลใหม่
3. หากยังเห็นของเก่า ให้ตรวจสอบว่า HTML อ้าง version ใหม่แล้ว
4. ลองรีเฟรชและเปิดลิงก์ใหม่ โดยไม่สรุปว่า push สำเร็จเท่ากับเว็บเปลี่ยน

ตำแหน่งอ้างอิงของงาน

Source: <https://github.com/Wiraphz/rain>

Website: <https://wiraphz.github.io/rain/>

เอกสาร PDF นี้เก็บไว้เฉพาะเครื่อง ไม่ได้ commit หรือ push

การแก้ครั้งต่อไปควรเลือก git add เฉพาะไฟล์เว็บที่ต้องการเผยแพร่

07 เตรียมอธิบายผลงานด้วยตัวเอง

ใช้ความเข้าใจและการสาธิตประกอบ แทนการอ้างว่าเขียนทุกส่วนเอง

ตัวอย่างคำอธิบายบทบาทที่ตรงกับกระบวนการ

“พอร์ตนี้พัฒนาโดยใช้ AI ช่วยเขียนโค้ดและแก้ภาพ ฉันเป็นผู้กำหนดข้อมูล รูปแบบที่ต้องการ และให้ข้อเสนอแนะระหว่างการปรับแก้ ส่วนการอ่านโค้ดและการทดสอบ ฉันจะอธิบายตามสิ่งที่ได้ลงมือจริง”
ควรปรับข้อความตามบทบาทจริงของผู้พูด ไม่ใช่แทนหลักฐานการลงมือ

คำถามที่ควรตอบได้

ทำไมต้องแยก content.js? เพื่อแก้ข้อมูลจุดเดียวและใช้ร่วมหลายหน้า
มือถือเปลี่ยนหน้าอย่างไร? CSS media query เปลี่ยนจำนวนคอลัมน์
ทำไมโลโก้ไม่ถูกตัด? ใช้ object-fit: contain แทน cover
กดเคียร์ติบัตรแล้วเกิดอะไร? เลือกข้อมูล แล้วเปิดภาพด้วย dialog
ทำไมออนไลน์ไม่อัปเดตทันที? ต้องรอ deploy และอาจมีแคชไฟล์เดิม
จำเป็นต้องมีฐานข้อมูลใหม่? เนื้อหาปัจจุบันไม่ต้องใช้ แต่ระบบใหม่อาจต้องมี

แบบฝึกหัดมือเพื่อยืนยันความเข้าใจ

1. เปลี่ยนข้อความหนึ่งค่าใน content.js แล้วชี้ว่าหน้าใดเปลี่ยนบ้าง
2. เพิ่มเคียร์ติบัตรตัวอย่างหนึ่งรายการ และอธิบายการจับคู่ชื่อไฟล์
3. ปรับระยะ gap ใน CSS แล้วเปรียบเทียบก่อนและหลัง
4. ลองย่อหน้าต่างผ่าน 700px และอธิบายกฎ CSS ที่ทำงาน
5. อ่าน git diff และสรุปว่าการแก้ันเปลี่ยนพฤติกรรมใด

หลักฐานที่ควรเก็บระหว่างเรียนรู้

ภาพก่อน-หลังพร้อมคำอธิบาย ปัญหาที่พบ วิธีแก้ และสิ่งที่ทดสอบจริง
ประวัติ commit ช่วยบอกว่าไฟล์เปลี่ยนเมื่อใด แต่ไม่ยืนยันว่าใครเขียนเอง
ทั้งหมด ควรใช้คู่กับการสาธิตและการอธิบายโค้ดด้วยความเข้าใจ

แหล่งข้อมูลหลัก: dist/*.html, content.js, app.js, styles.css,
server.cjs, validate.cjs และประวัติ Git ของโครงการ RXIN